

Claude Code — Working Reference

ShadesCaddie / Plans2Putts (P2P) project · distilled from the official Claude Code documentation (code.claude.com/docs) and session working notes · June 2026

Purpose

A single reference for KT and the navigator chat to review before advising or building with Claude Code. Claude Code itself auto-reads **CLAUDE.md** at the repo root each session; this document is the deeper background behind that file. Keep CLAUDE.md short (under ~200 lines); keep the detail here.

1. The most important feature: CLAUDE.md

A file named **CLAUDE.md** at the repo root is read automatically at the start of **every** Claude Code session. It is the built-in "project brain" — no ramp-up paste needed each time.

- Put standing context and rules here: what the project is, conventions, key commands, "always do / never do" rules.
- Keep it under ~200 lines. It loads into context every session, so bloat costs tokens and reduces how well Claude follows it. Move long reference material to skills or separate docs.
- Editing CLAUDE.md mid-session does NOT take effect until `/clear`, `/compact`, or restart — it is read once at session start.
- Run `/memory` inside a session to view or edit it.
- Also works at `.claude/CLAUDE.md` if you prefer to keep the root clean. A personal, gitignored `CLAUDE.local.md` can hold private preferences.

Related: `.claude/rules/*.md` hold topic-scoped instructions (optionally gated to file paths via `paths: frontmatter`). `skills/` hold reusable workflows you invoke with `/name`. Start with just CLAUDE.md; add the rest only when a clear need appears.

2. Where Claude Code runs (surfaces)

- **Desktop app (what KT uses):** launch Claude, click the **Code** tab. Review diffs visually, run sessions side by side. A paid subscription is required.
- **Web — no install:** `claude.ai/code` runs in a browser; also works in the Claude iOS app. This is the fallback if the laptop is ever unavailable (avoids installing anything).
- **Terminal CLI, VS Code, JetBrains:** other surfaces; not needed for now.
- All surfaces share the same engine, so CLAUDE.md, settings, and MCP work across all of them. The project lives in GitHub regardless of surface — nothing is trapped on one machine.

Note on install: native installers exist for macOS/Linux/Windows (npm install is deprecated). KT cannot install on a company-owned laptop, but the Desktop app already works there; the web version needs no install at all.

3. Core working principles

- **Orient before building.** Have Claude Code read the repo and summarize the project and the task back to you BEFORE it writes code. Catches misunderstandings cheaply.
- **One contained task at a time.** e.g. "build Step 0 + Step 1 for Illinois only," not "build the whole pipeline."
- **Be specific.** "Fix the login bug where users see a blank screen after wrong credentials" beats "fix the bug."
- **Make it show its plan** before big executions, and review it.
- **Review what it commits.** It writes to your repo; glance at the diffs. It asks permission before modifying files (approve each, or enable "accept all" for a session; Shift+Tab cycles permission modes).
- **Keep the brief as source of truth.** When it drifts, point it back to CLAUDE-CODE-BRIEF-data-pipeline.md.

4. The context window (why focus matters)

Everything Claude Code reads — files, command output, your messages — fills a finite context window. Large file reads dominate usage. Being specific means fewer reads, which keeps the session healthy and responses sharp.

- `/context` — see current usage by category.
- `/clear` — reset the conversation when switching to unrelated work.
- `/compact` — summarize a long session to free space. Run it at a natural break between tasks, not mid-task. You can focus it: `/compact focus on the data pipeline`.
- **Subagents:** for research-heavy side tasks, a subagent works in its own separate context and returns only a summary — keeping your main session clean.

What survives a compaction: the system prompt, root `CLAUDE.md`, and auto-memory are re-loaded from disk. Path-scoped rules and nested `CLAUDE.md` files are summarized away until a matching file is read again. Put anything that must always persist in the root `CLAUDE.md`.

5. A few mechanics worth knowing

- **Prompt caching is automatic.** Switching models or effort level mid-session forces a slower, more expensive "uncached" turn. Pick your model at the start and avoid switching mid-task.
- **Auto memory:** Claude Code can keep its own notes across sessions (build commands, patterns it learned). On by default; toggle with `/memory`.
- **Git is conversational:** "commit my changes with a descriptive message," "create a branch," "show me the last 5 commits" all work in plain language.
- **Work from anywhere:** a session can move between laptop, web, and phone; you can hand a session to the Desktop app or pull a web/iOS task into the terminal.

6. Useful commands

Command	What it does
<code>/help</code>	List available commands
<code>/memory</code>	View / edit <code>CLAUDE.md</code> and auto memory
<code>/context</code>	Show current context-window usage
<code>/clear</code>	Clear conversation (between unrelated tasks)
<code>/compact</code>	Summarize a long session to free space
<code>/resume</code>	Continue a previous conversation
<code>/login</code>	Switch or re-authenticate account

7. Our working model (KT + two Claudes)

- **Claude Code = the BUILDER.** Reads, writes, runs, tests, and commits real files. Better at hands-on coding than the navigator and sees the actual repo.
- **Navigator chat (claude.ai) = the NAVIGATOR.** Holds strategy and project history, pushes back on plans, helps phrase instructions, and translates errors. It is sealed off from the repo and works via KT relaying.
- **The brief is the bridge.** CLAUDE-CODE-BRIEF-data-pipeline.md encodes the strategy into instructions Claude Code can follow.
- **Keep the navigator in the loop for decisions that matter** — what data to collect, whether something is legal, whether you're building the right thing. A pure executor can build the wrong thing well.

8. The one hard rule for this project

Never fabricate GPS coordinates. The rangefinder needs per-hole coordinates (tee, green front/center/back, hazards). No free or cheap source produces these — the Census geocoder gives one property point, scorecards give par/yardage only, and satellite imagery is a picture, not coordinates. Real coordinates come only from an iGolf license, a golfer's on-course GPS capture, or careful human satellite pin-placement. Unknown coordinates stay NULL with status "unverified." A wrong coordinate means a wrong yardage on a real shot — the one failure this product cannot have. The data pipeline builds a course DIRECTORY (name, address, geocode, scorecard), not working distances.

Reference docs in the repo (read alongside this):

- **PROJECT-STATUS.md** — current state, decisions, lessons, to-do list (start here).
- **COURSE-DATA-STRATEGY.md** — the full course-data + growth strategy.
- **CLAUDE-CODE-BRIEF-data-pipeline.md** — the spec for the data-pipeline task.
- **CLAUDE.md** — the auto-loaded short instructions (this PDF is the background behind it).

Source: the official Claude Code documentation at code.claude.com/docs (Overview, Quickstart, Extend Claude Code, the .claude directory, Explore the context window, and Prompt caching), plus ShadesCaddie session working notes. The live docs reflect the most current state if features change over time.

9. Platforms, integrations & automation

Beyond the core surfaces, Claude Code connects to a range of tools and can automate parts of the development workflow. The items below are useful to know exist; most are NOT needed for ShadesCaddie's current stage (a static web app + a data pipeline) and can be ignored until a specific need arises.

Integrations

- **CI/CD automation:** GitHub Actions and GitLab CI/CD can run Claude in a pipeline for automated PR creation, code review, and issue triage.
- **Collaboration:** Slack and automated Code Review integrations support team workflows and PR checks (mention @Claude in Slack to route a bug report toward a fix).
- **Browser:** a Chrome integration enables live web-app testing, debugging, and form automation alongside coding tasks.
- **Enterprise hosting:** runs via the Claude API, Amazon Bedrock, or Google Vertex AI for data-residency and procurement needs.

Desktop app capabilities

- Visual diff review and running multiple sessions side by side.
- Scheduled / recurring tasks (e.g. morning PR reviews, dependency audits, CI status summaries) via Routines or Desktop scheduled tasks.
- Remote development over SSH — drive a remote environment from the Desktop UI.
- GUI automation ("computer use") on macOS/Windows for testing native apps or driving tools that have no API.

Automated code review & security

- **Code Review service:** analyzes GitHub PRs and posts inline findings (logic errors, security issues, edge cases, regressions); a fleet of agents reviews with full-codebase context, then a verification step filters false positives. Configurable per org/repo; cost scales with PR size.
- **Security review:** a plugin can review Claude's own changes for vulnerabilities in the same session — pattern matching on each edit, model review after each turn, deeper review on commit. Complements /security-review and CI scanning.
- Both align to project standards via **CLAUDE.md** (and optional REVIEW.md / security-patterns.yaml files). They assist human review, not replace it.

Best practices for integrations & CI/CD

- Make Claude respect project conventions defined in CLAUDE.md.
- Never commit secrets; use secure credential mechanisms (environment variables, secrets stores).
- Keep CLAUDE.md concise and targeted.
- Manage cost with targeted commands and concurrency limits.
- Always review AI-generated changes before merging.

Source: the official Claude Code documentation from Anthropic. As with any documentation, the live docs at code.claude.com/docs reflect the most current state if a feature changes over time.